



Tu primer endpoint REST con Spring Boot

Serie: Java con Spring Boot de 0 a Pro • Video #2
APIs REST con Spring Boot

¿Qué aprenderás hoy?

01

Controladores REST

Qué es un `@RestController` y cómo funciona en el ecosistema de Spring Boot

02

Endpoint básico

Crear tu primer endpoint GET con `@GetMapping` y retornar JSON

03

Ejecución local

Arrancar la aplicación y revisar los logs de inicio

04

Pruebas con Postman

Probar tu endpoint y validar las respuestas HTTP



Prerrequisitos: Video 1 (REST y HTTP) + Playlist A (Fundamentos de Spring Boot)



Recordatorio: ¿Qué es un endpoint REST?

URL específica

Una dirección web que recibe peticiones HTTP (GET, POST, PUT, DELETE)

Procesamiento

Ejecuta lógica de negocio y maneja datos según el método HTTP

Respuesta JSON

Retorna datos estructurados que el cliente puede consumir

Los endpoints son los **puntos de entrada** de tu API REST. Cada endpoint tiene una responsabilidad específica y maneja un tipo de operación.

@RestController: El corazón de tu API

Anotación especializada

Combina @Controller +
@ResponseBody automáticamente

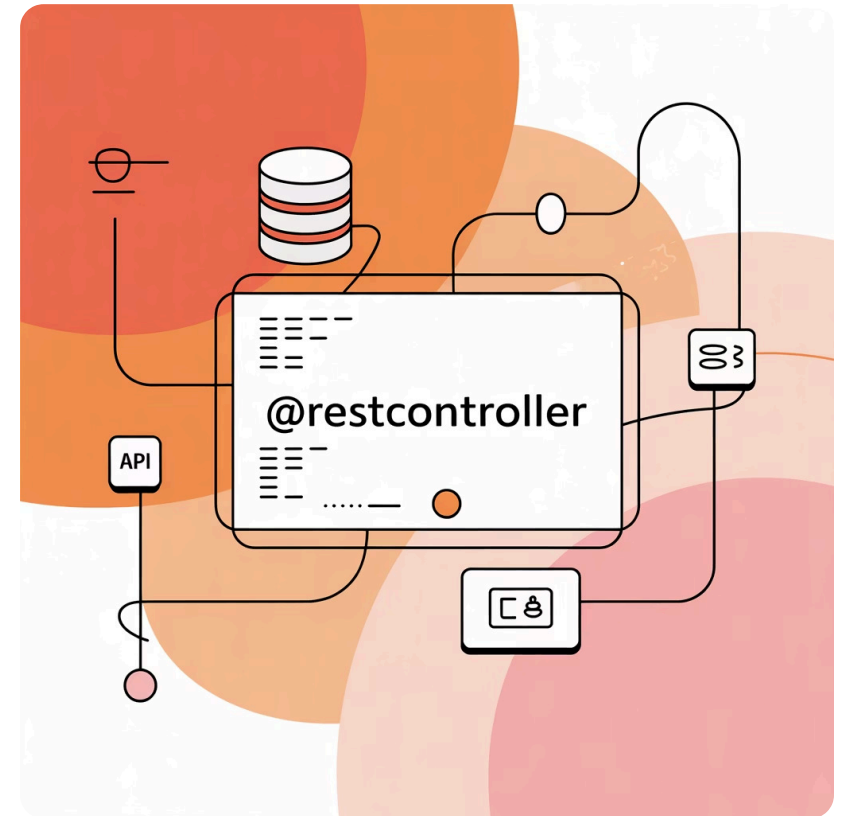
Conversión automática

Spring convierte objetos Java a
JSON sin configuración extra

Manejo de HTTP

Gestiona peticiones y respuestas
HTTP de forma transparente

¿Por qué @RestController? Simplifica el desarrollo de APIs REST eliminando la necesidad de anotar cada método con @ResponseBody.



Spring Boot getmapping annotation example code.

@GetMapping: Definiendo rutas GET



Mapeo de URL

Define qué URL activará tu método: `@GetMapping("/saludo")`



Método HTTP GET

Específico para operaciones de lectura y consulta de datos



Respuesta automática

El valor retornado se convierte automáticamente a JSON

Con **@GetMapping** defines de forma declarativa cómo tu aplicación responde a peticiones HTTP GET específicas.

Snippet: Tu primer endpoint

```
@RestController
public class SaludoController {

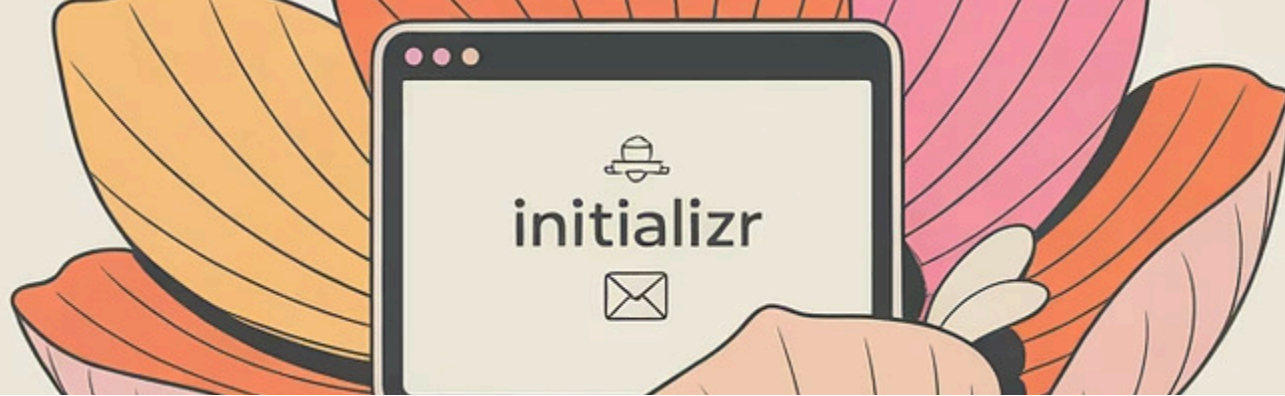
    @GetMapping("/saludo")
    public String obtenerSaludo() {
        return "¡Hola desde Spring Boot!";
    }

    @GetMapping("/usuario")
    public Map<String, String> obtenerUsuario() {
        Map<String, String> usuario = new HashMap<>();
        usuario.put("nombre", "Juan");
        usuario.put("rol", "desarrollador");
        return usuario;
    }
}
```

Este código crea **dos endpoints**:

- **GET /saludo** → Retorna texto plano
- **GET /usuario** → Retorna JSON con datos del usuario

Spring Boot maneja automáticamente la serialización y el Content-Type apropiado.



Checklist para la demostración

1

Spring Initializr

Crear proyecto con dependencies: Spring Web

- JDK 17+
- Spring Boot 3.x
- Maven como build tool

2

IDE Setup

Importar proyecto en tu IDE favorito

- IntelliJ IDEA
- Eclipse
- VS Code con extensiones Java

3

Postman

Herramienta para testing de APIs

- Crear nueva collection
- Configurar requests GET
- Validar responses

[Demostración en vivo]

Ahora vamos a crear nuestro primer endpoint REST paso a paso

Crear proyecto

Spring Initializr + IDE

Escribir código

@RestController + @GetMapping

Ejecutar app

mvn spring-boot:run

Probar en Postman

GET localhost:8080/saludo



Resumen: Lo que aprendiste



@RestController

Anotación que combina @Controller + @ResponseBody para crear APIs REST



@GetMapping

Define endpoints GET que responden a URLs específicas



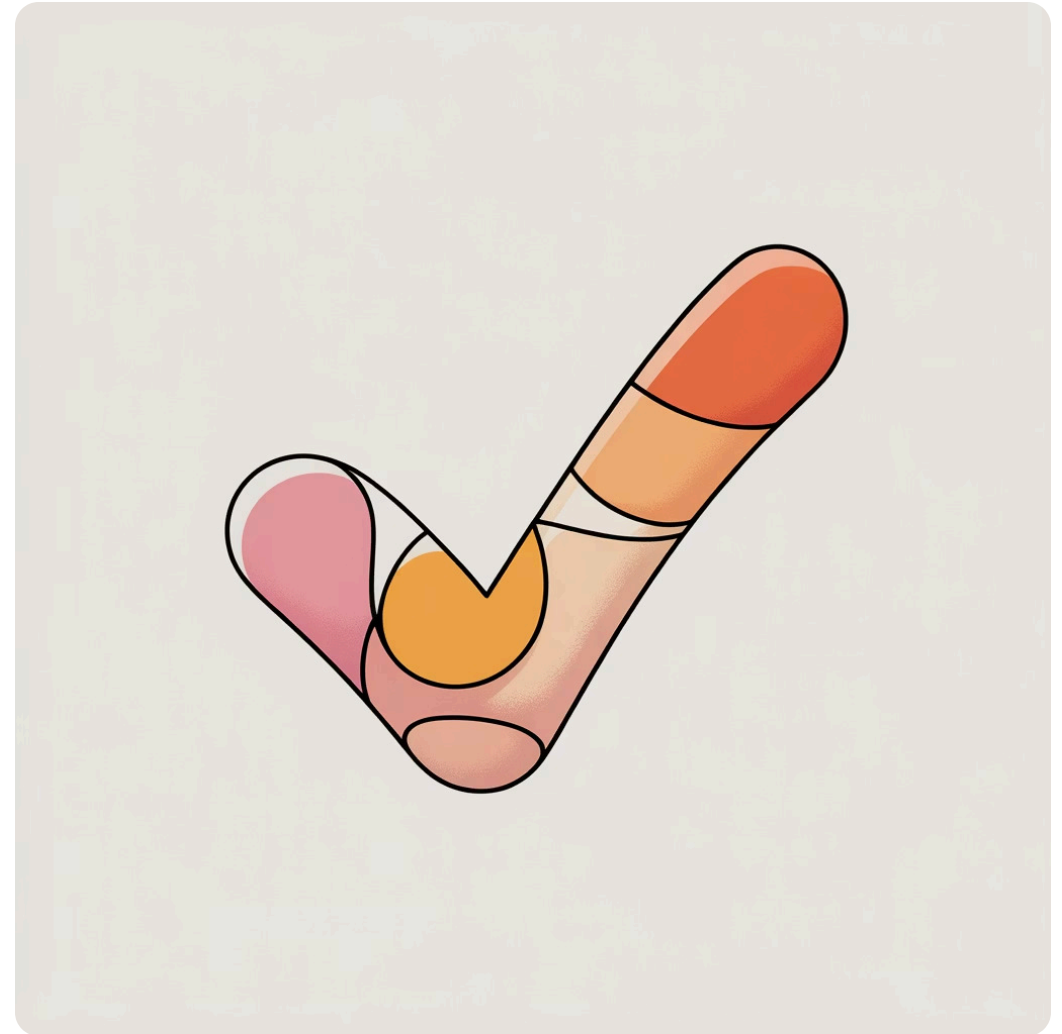
JSON automático

Spring Boot serializa automáticamente objetos Java a JSON



Testing con Postman

Validar respuestas HTTP y estructura JSON de tus endpoints



¡Felicidades! Ya tienes las bases para crear APIs REST robustas con Spring Boot.



¡Éxito en tu camino REST!

Has dado un paso crucial en tu desarrollo con Spring Boot.
Sigue construyendo, sigue aprendiendo.